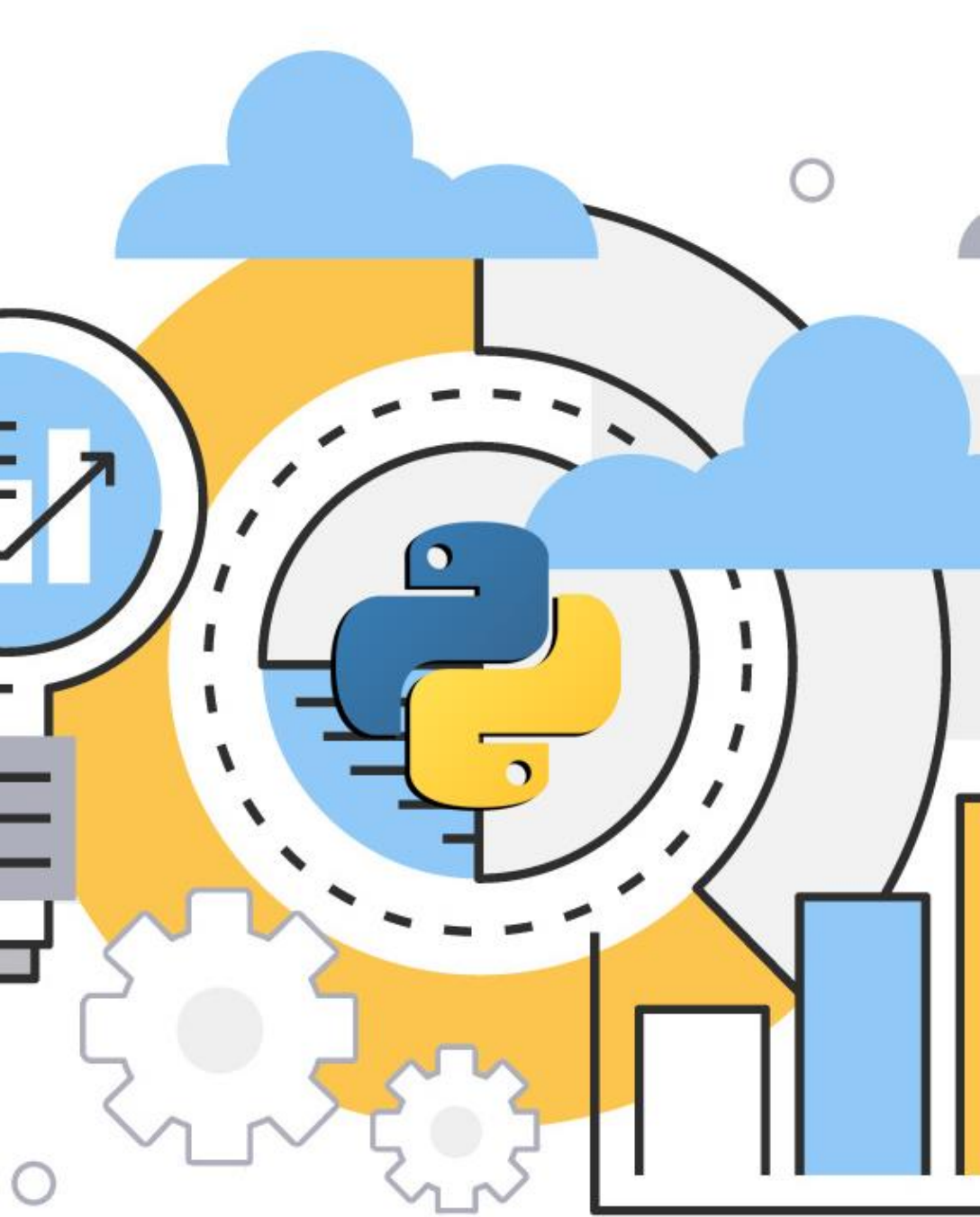


CIÊNCIA DE DADOS COM PYTHON

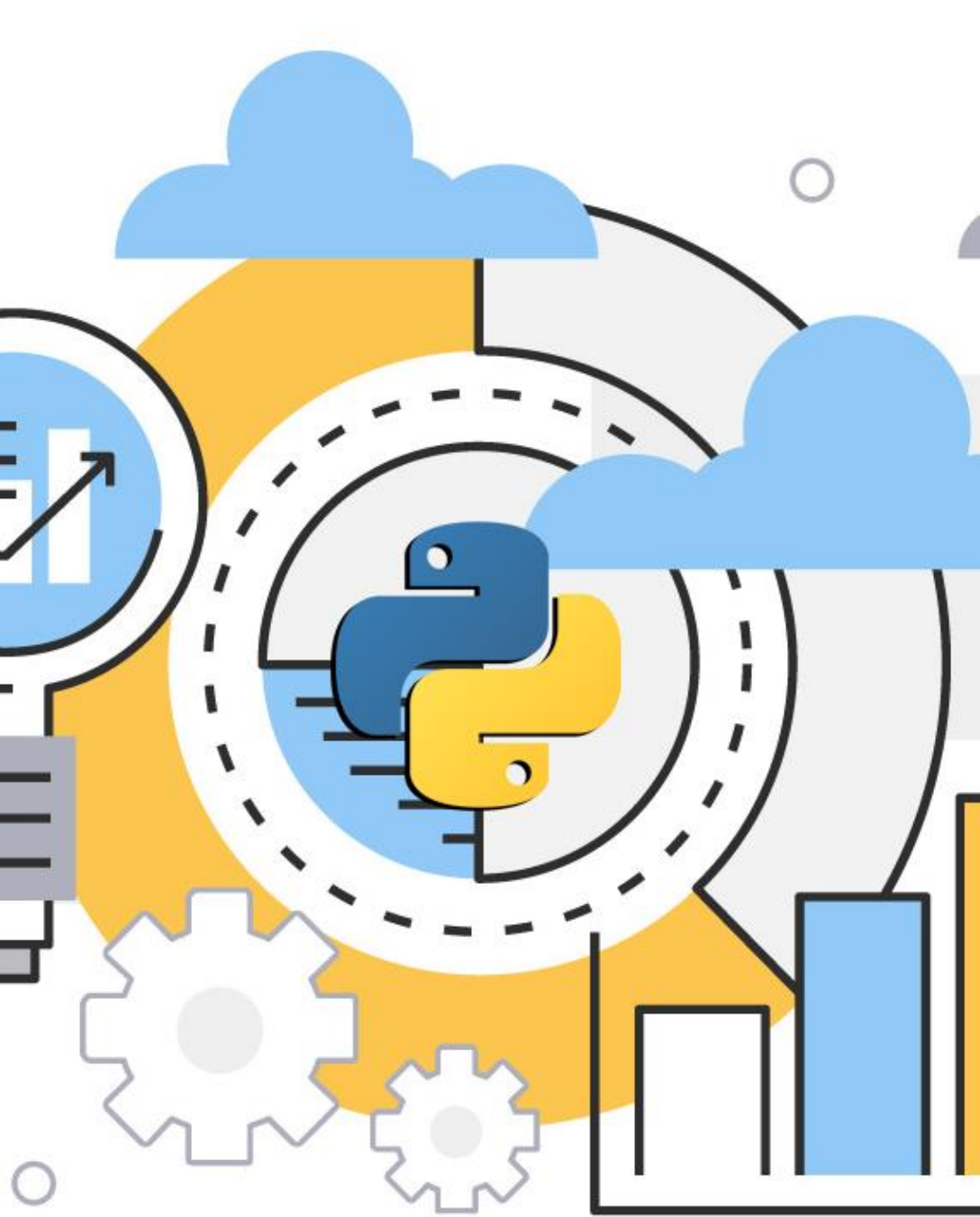
CAP.7 - INTRODUÇÃO ÀS SÉRIES TEMPORAIS

Prof. Renzo Paranaíba Mesquita



OBJETIVOS

- Compreender o que são Séries Temporais e sua importância na Ciência de Dados;
- Entender o que são Tendências, Sazonalidades e Ciclos e como eles nos ajudam a compreender o comportamento dos dados ao longo do tempo;
- Compreender o que é Previsão e como podemos utilizar de técnicas poderosas para prevermos valores futuros de Séries Temporais;
- Aplicarmos este conceito por meio da linguagem Python.

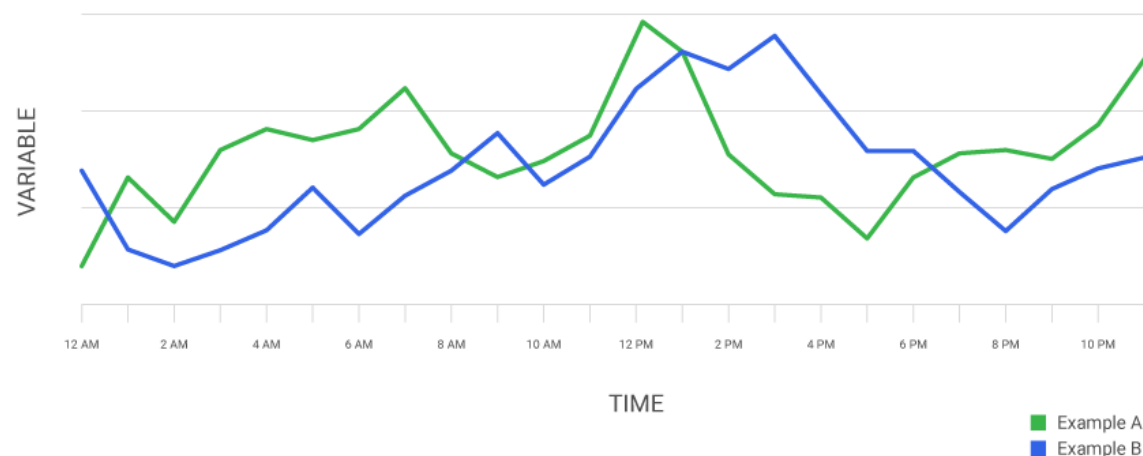


TÓPICOS

1. Introdução
2. Componentes de uma Série Temporal
3. Importando Dados e Visualizando Séries Temporais
4. Decomposição de Séries Temporais
5. Previsão em Séries Temporais

7.1. INTRODUÇÃO

- Séries temporais são dados coletados em intervalos de tempos regulares (como segundos, hora, dias, meses, etc.)
- Elas possuem uma dependência da ordem dos dados, e os intervalos precisam ser regulares para isso ser possível;
- Estudamos séries temporais quando queremos compreender como um fenômeno ocorre ao longo do tempo;
- Além disso, conseguimos até mesmo prever eventos futuros baseado em fenômenos passados, utilizando de técnicas matemáticas.



7.1. INTRODUÇÃO

- Para ser considerado uma Série Temporal, deve-se seguir a seguinte regra:

- **1) MEDIDA -> 2) FATO -> 3) UNIDADE DE TEMPO**

- **EXEMPLOS DE SÉRIES TEMPORAIS:**

- Cotação da bolsa de valores por minuto;

Cotação (Medida) da bolsa de valores (Fato) por minuto (Unidade de Tempo)

- Internações de pacientes por dia;

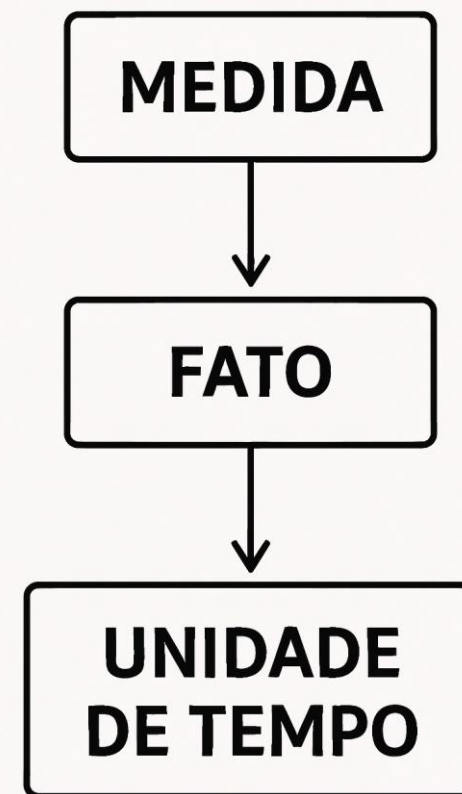
Internação (Medida) de pacientes (Fato) por dia (Unidade de Tempo)

- Investimento em educação por ano;

Investimento (Medida) em educação (Fato) por ano (Unidade de Tempo)

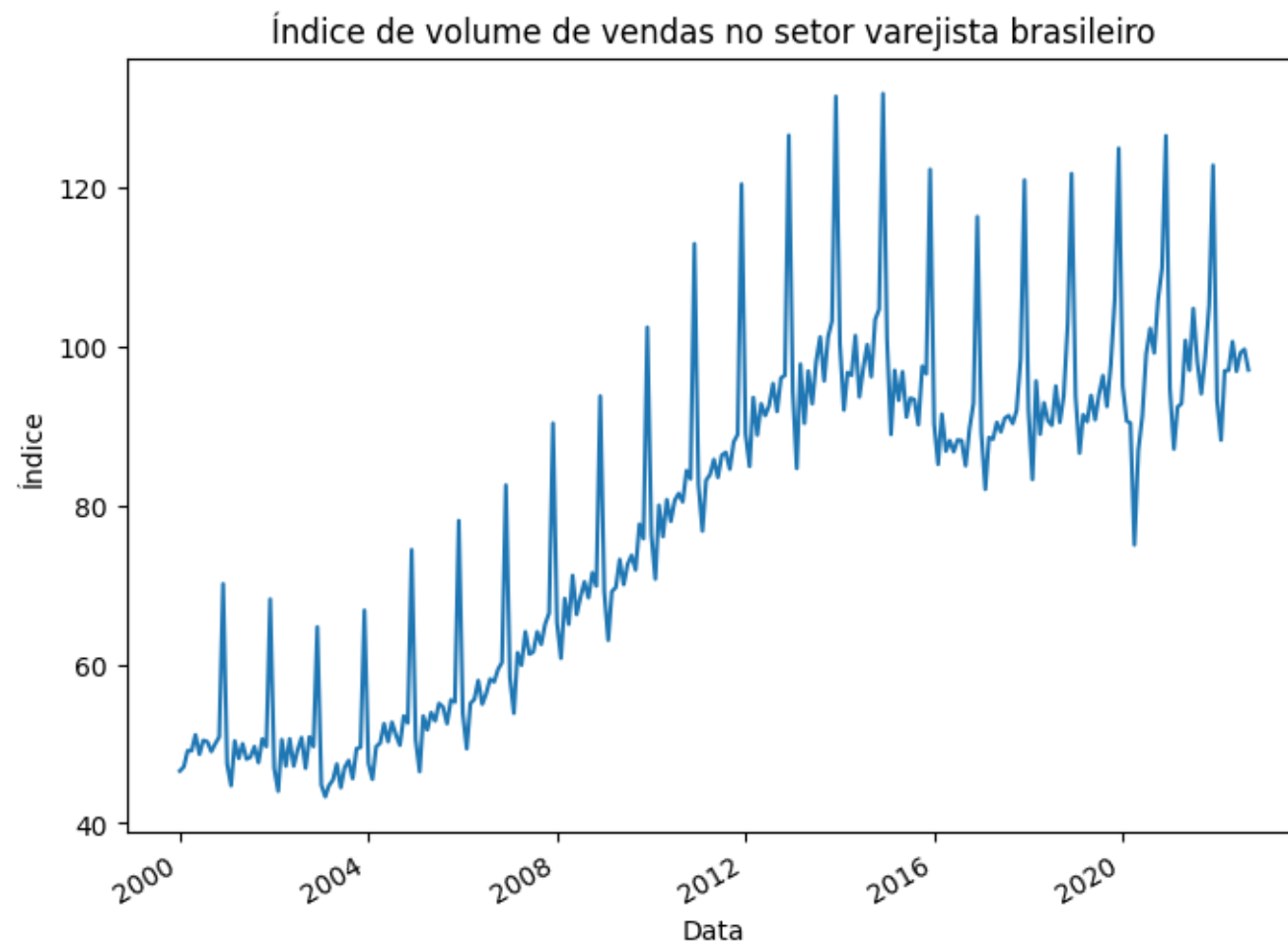
- **O QUE NÃO SÃO SÉRIES TEMPORAIS:**

- Cadastro de clientes;
- Salários de pessoas que visitam uma loja;
- Renda per capita de um país;
- Etc.



7.1. INTRODUÇÃO

- Toda Série Temporal tem um gráfico, que ilustra a janela de tempo em que os dados se encontram.
- Ex: Índice do volume de vendas no setor varejista brasileiro

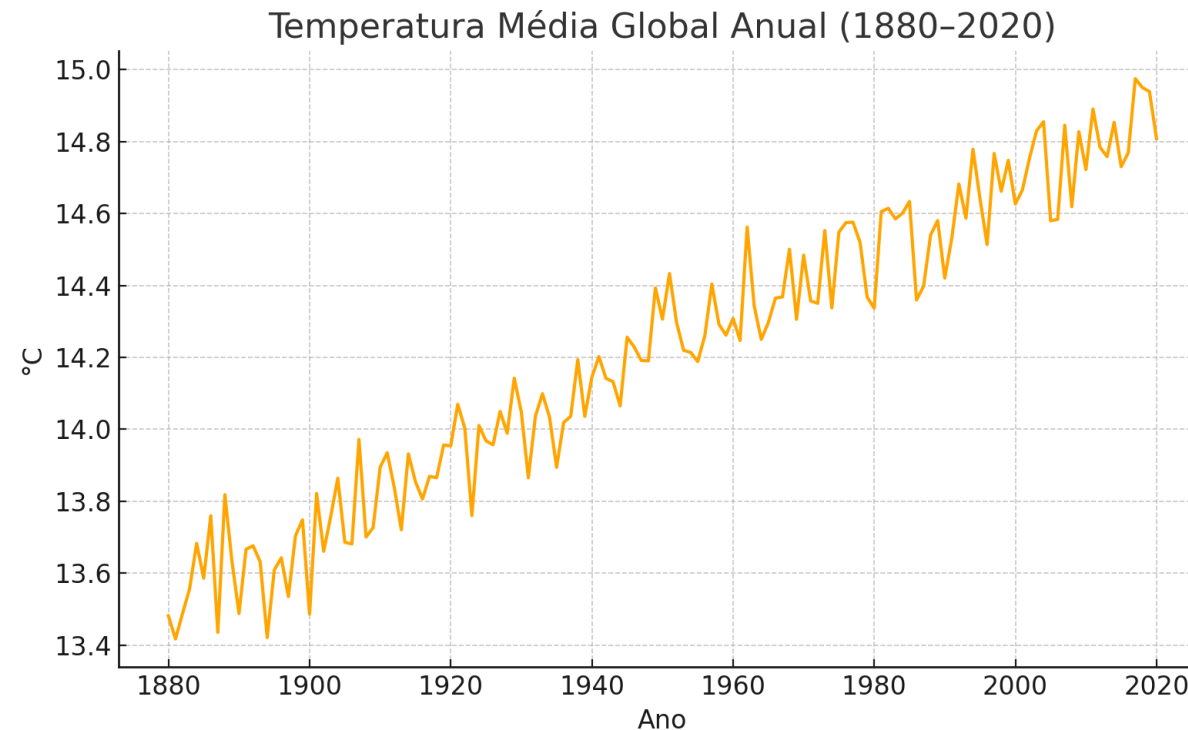


7.2. COMPONENTES DE UMA SÉRIE TEMPORAL

- Os principais componentes de uma série temporal são:

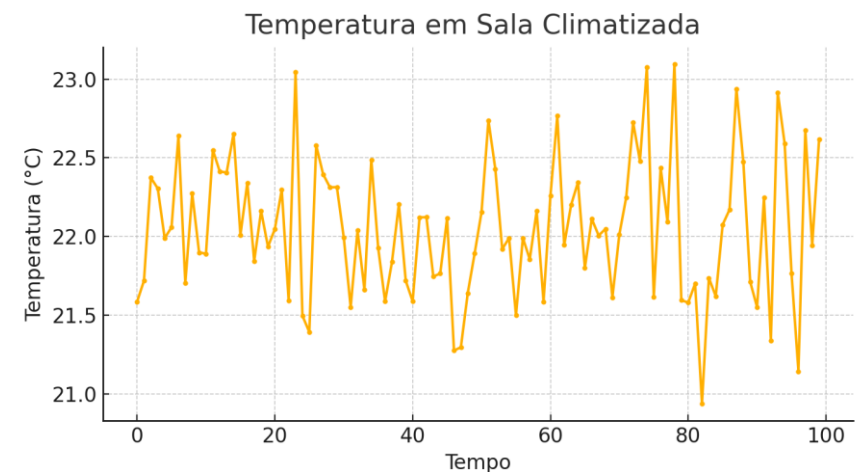
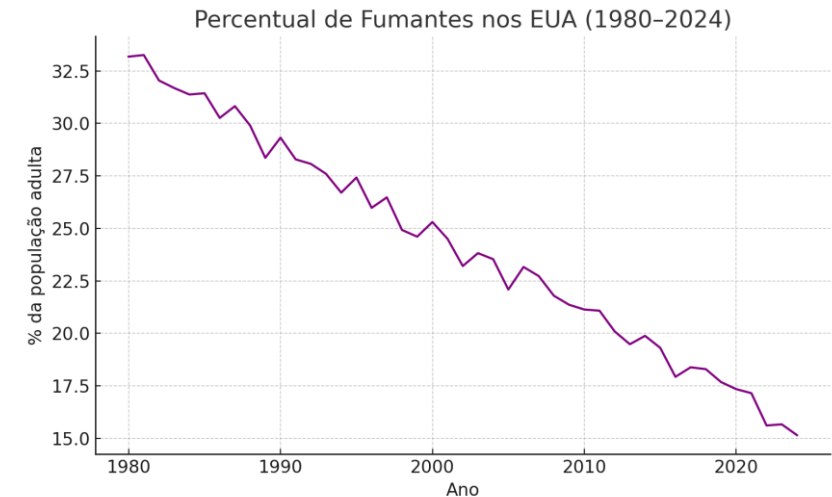
1. TENDÊNCIA: REPRESENTA A DIREÇÃO DE LONGO PRAZO DOS DADOS. ELA PODE SER:

- A) CRESCENTE: valores aumentando com o tempo;
- Ex: Temperatura Média Global Anual



7.2. COMPONENTES DE UMA SÉRIE TEMPORAL

- **B) DECRESCENTE:** valores diminuindo com o tempo;
- **Ex: Percentual de Fumantes nos EUA**
- **C) ESTACIONÁRIA:** valores sem tendência clara.
- **Ex: A temperatura em uma sala climatizada**
- Uma série com tendência estacionária possui média e variância constantes ao longo do tempo, sem tendência de crescimento ou queda.

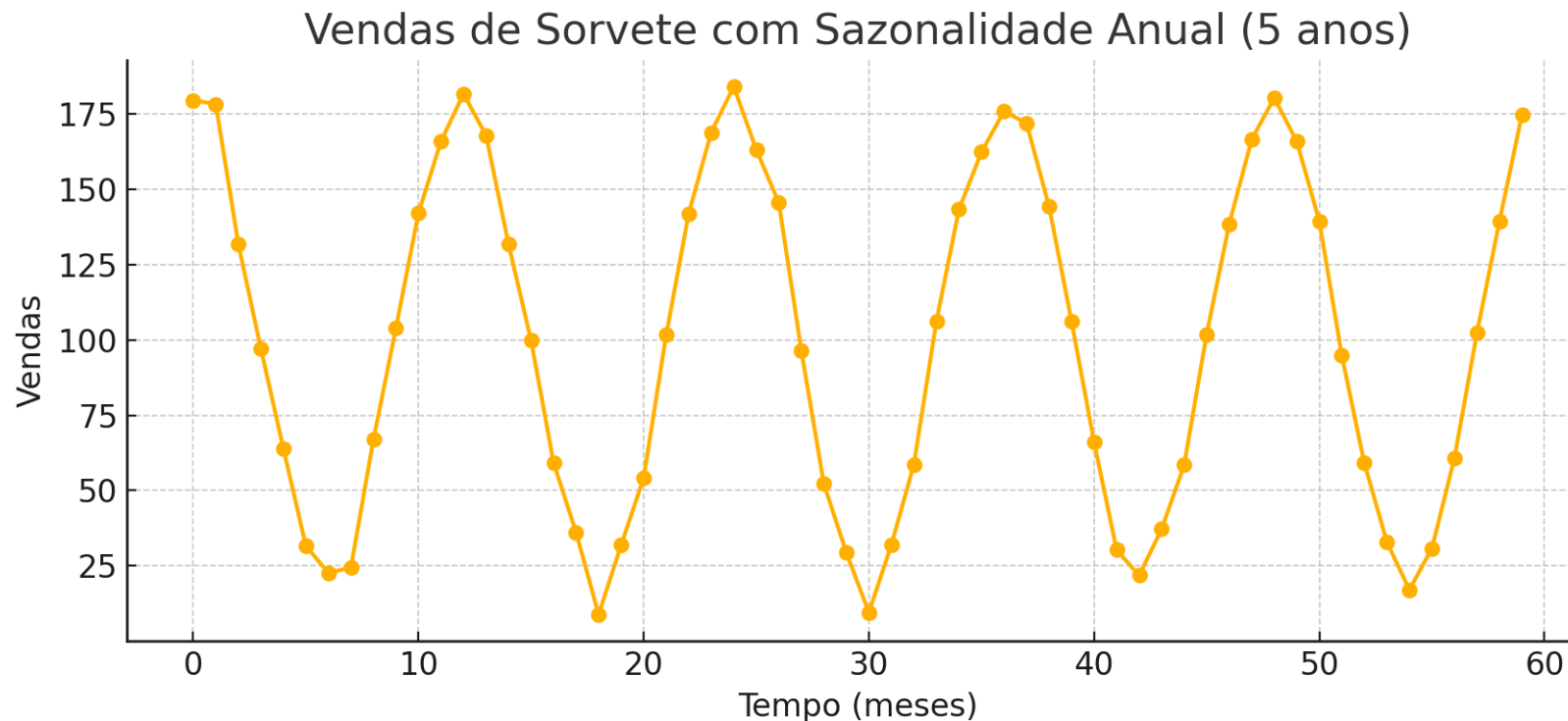


7.2. COMPONENTES DE UMA SÉRIE TEMPORAL

- Os principais componentes de uma série temporal são:

2. SAZONALIDADE: REPRESENTA VARIAÇÕES QUE OCORREM EM INTERVALOS REGULARES, GERALMENTE CAUSADAS POR FATORES SAZONAIS COMO ESTAÇÕES DO ANO, MESES OU DIAS.

- Ex: Vendas de sorvete com Sazonalidade Anual (pico de vendas no verão e queda clara no inverno)

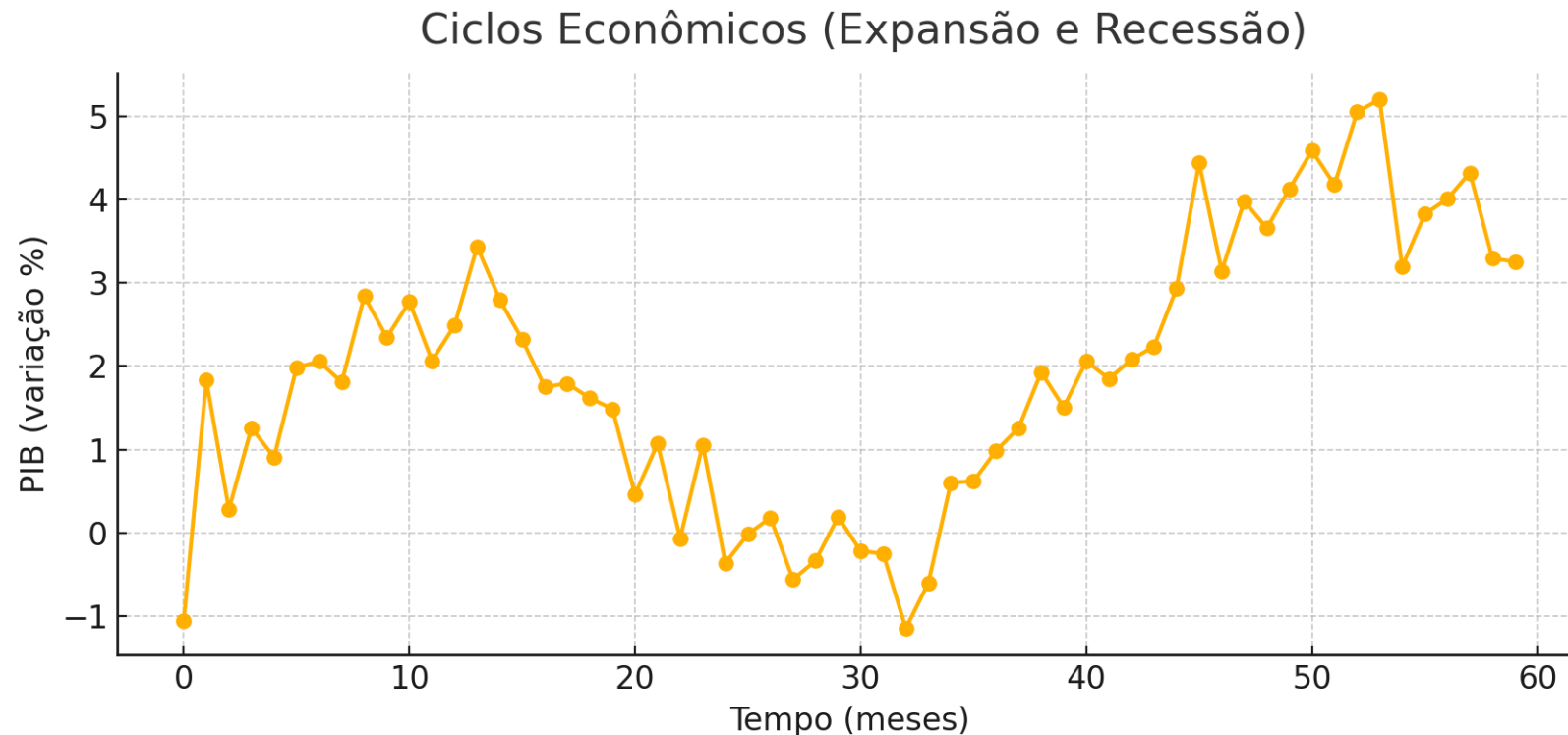


7.2. COMPONENTES DE UMA SÉRIE TEMPORAL

- Os principais componentes de uma série temporal são:

3. CICLO: REPRESENTA FLUTUAÇÕES DE LONGO PRAZO QUE NÃO SEGUE UM PADRÃO FIXO COMO A SAZONALIDADE. SÃO GERALMENTE ASSOCIADAS A EVENTOS EXTERNOS.

- Ex: Alternância entre crescimento e queda da economia ao longo do tempo (expansão e recessão)

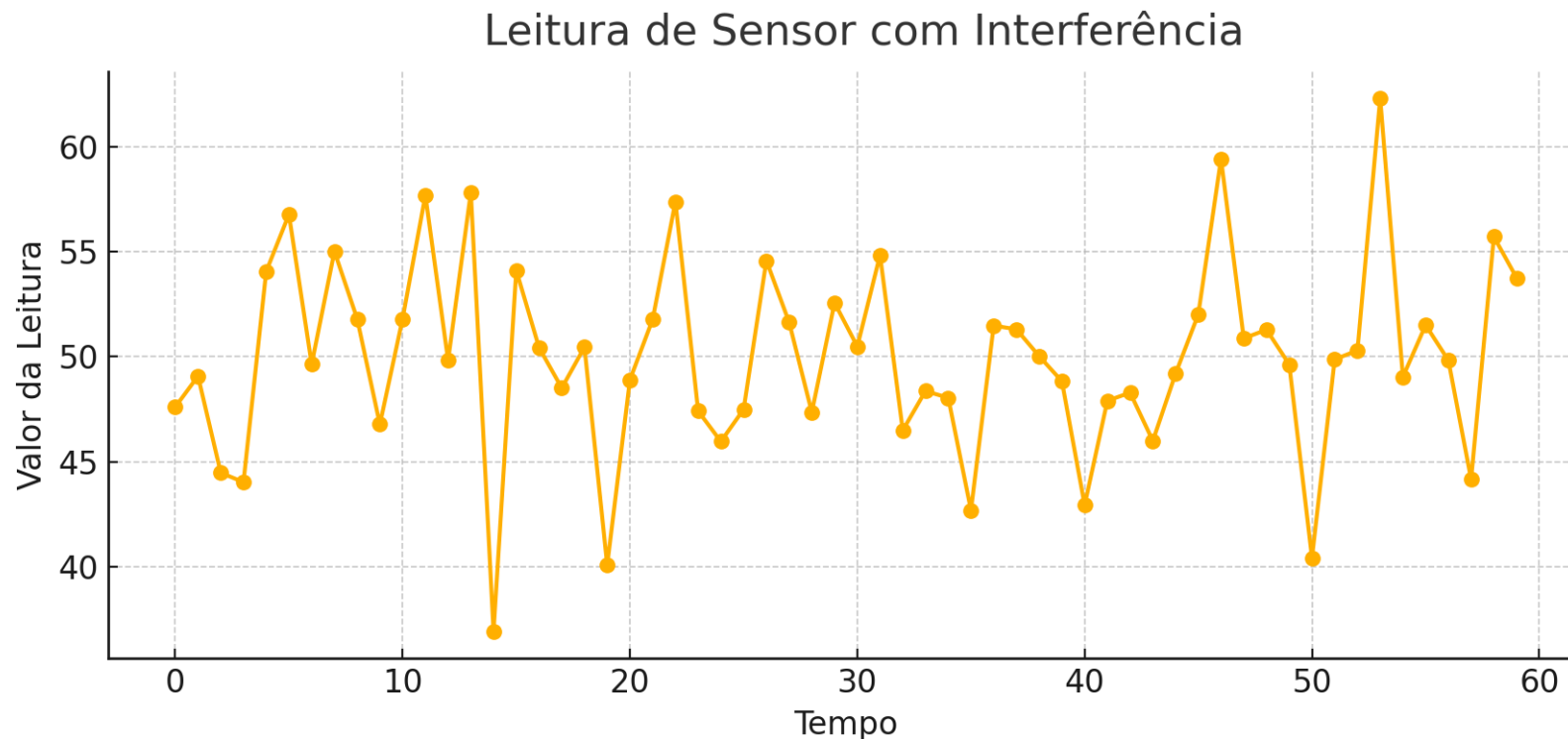


7.2. COMPONENTES DE UMA SÉRIE TEMPORAL

- Os principais componentes de uma série temporal são:

4. RUÍDO (RESÍDUO OU ALEATORIEDADE): COMPONENTE IMPREVISÍVEL DE UMA SÉRIE. CAUSADO POR VARIAÇÕES ALEATÓRIAS OU FATORES QUE NÃO PODEM SER EXPLICADOS.

- Ex: Leitura de um sensor com interferência



7.3. IMPORTANDO DADOS E VISUALIZANDO SÉRIES TEMPORAIS

- Faremos uso do Python e suas bibliotecas NumPy, Pandas e Matplotlib para plotarmos e analisarmos Séries Temporais;

- **EXEMPLOS:**

```
import pandas as pd  
import numpy as np
```

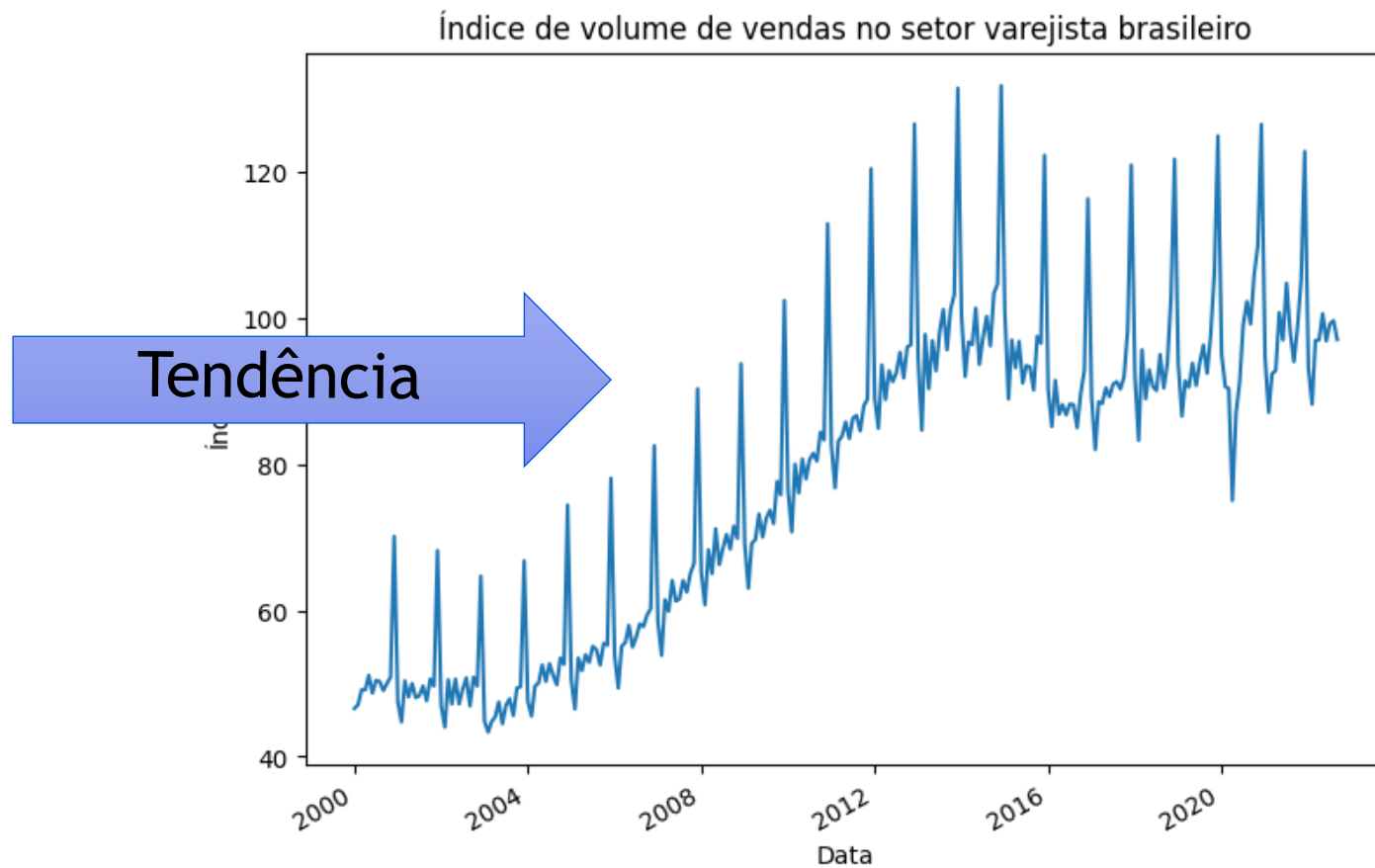
```
# Carregando um dataset do Índice de Volume de Vendas no Setor Varejista Brasileiro  
dataset = pd.read_csv('retail_index_br.csv', delimiter=';', index_col='date',  
parse_dates=True)
```

```
# Transformando os dados de y em float  
dataset['retail_index'].astype(float)
```

```
# Plotando a Time Series  
dataset['retail_index'].plot(figsize=(8, 6),  
title='Índice de volume de vendas no setor varejista brasileiro',  
xlabel='Data', ylabel='Índice', x_compat=True)
```

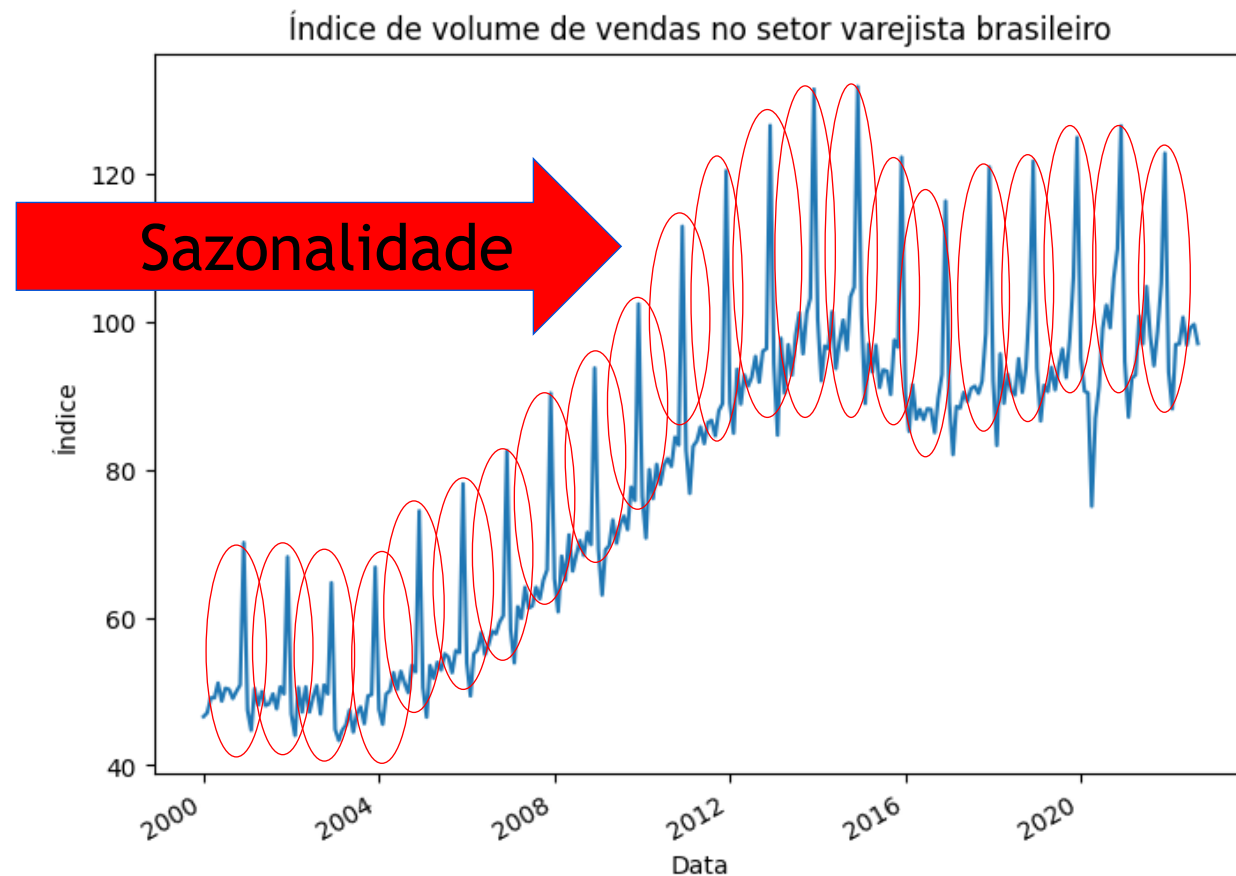
7.3. IMPORTANDO DADOS E VISUALIZANDO SÉRIES TEMPORAIS

- Observe que uma Série Temporal **pode** possuir um ou mais componentes:



7.3. IMPORTANDO DADOS E VISUALIZANDO SÉRIES TEMPORAIS

- Observe que uma Série Temporal **pode** possuir um ou mais componentes:



7.4. DECOMPOSIÇÃO DE SÉRIES TEMPORAIS

- A decomposição é o processo de separar a Tendência (Trend), Sazonalidade (Seasonal) e Ruído (Resid) de uma Série Temporal;
- Ela ajuda a melhor compreender o comportamento dos dados ao longo do tempo;

- **EXEMPLOS:**

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
```

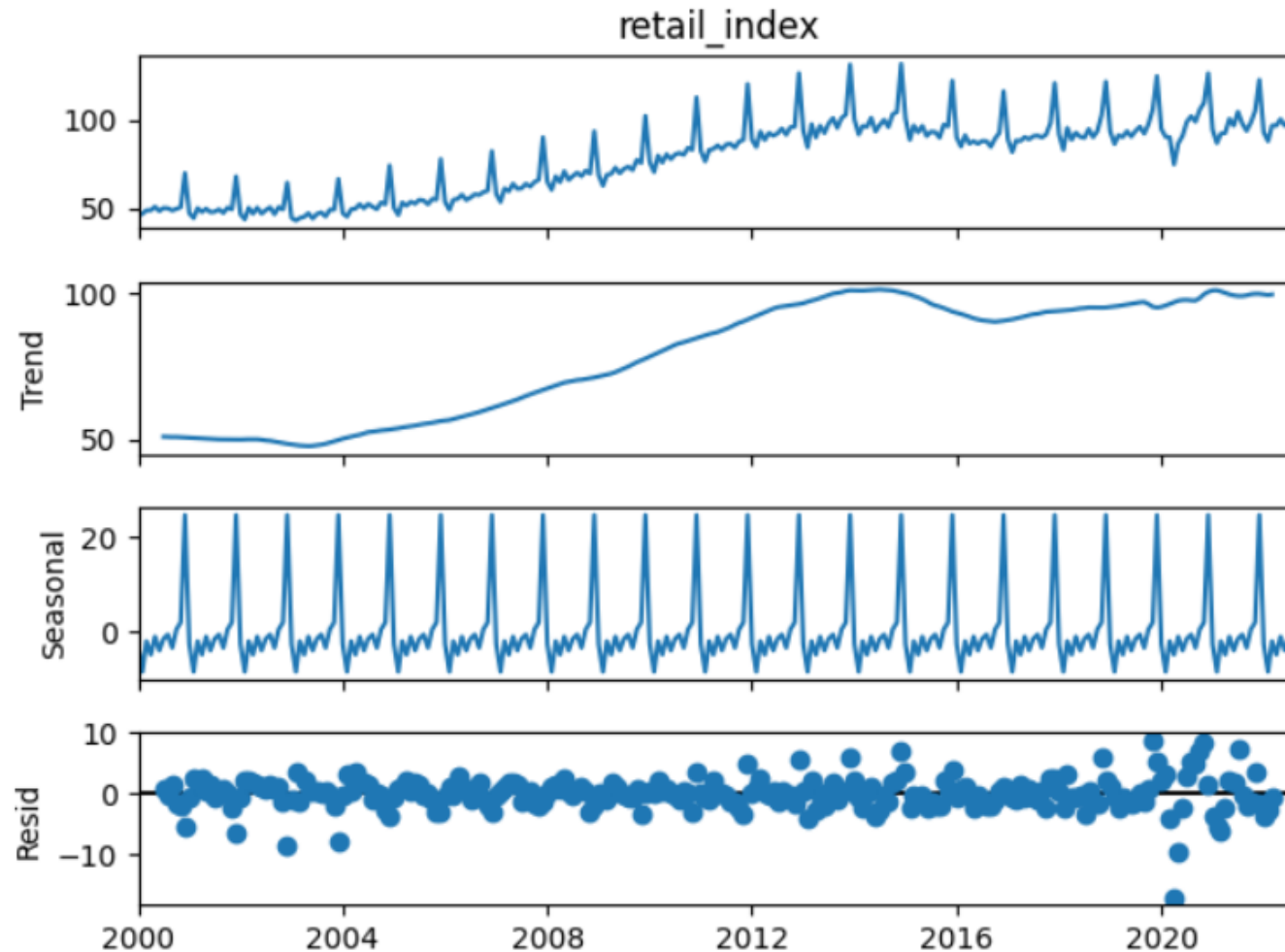
```
# Carregando o dataset do Índice de Volume de Vendas no Setor Varejista Brasileiro
dataset = pd.read_csv('retail_index_br.csv', delimiter=';', index_col='date', parse_dates=True)
```

```
# Decomposição aditiva
decomposition = seasonal_decompose(dataset['retail_index'], model='additive')
```

```
# Plotando os componentes
decomposition.plot()
plt.tight_layout()
```

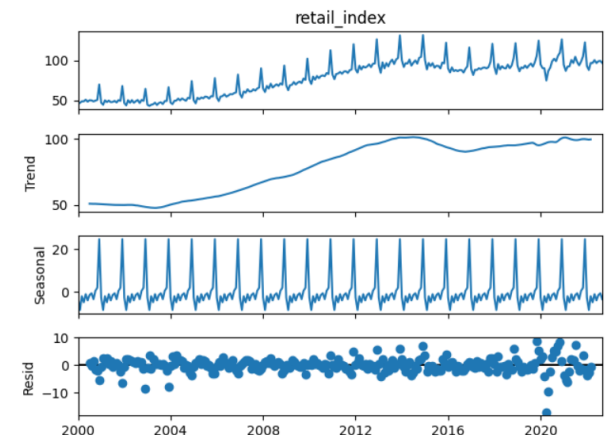

7.4. DECOMPOSIÇÃO DE SÉRIES TEMPORAIS

- Observe como a decomposição facilita a identificação dos componentes.



7.4. DECOMPOSIÇÃO DE SÉRIES TEMPORAIS

- No exemplo anterior, utilizamos de uma decomposição **aditiva**, porém, também podemos utilizar de uma decomposição **multiplicativa**;
- Use **decomposição aditiva** quando:
 - A amplitude da sazonalidade é constante ao longo do tempo (ou seja, a variação sazonal não cresce nem diminui com a tendência);
 - **Exemplo: temperatura média mensal — flutuações sazonais relativamente estáveis ao longo dos anos, independentemente da tendência;**
- Use **decomposição multiplicativa** quando:
 - A sazonalidade varia proporcionalmente à tendência (ou seja, os picos e vales sazonais ficam maiores ou menores à medida que a série cresce ou decresce);
 - **Exemplo: vendas mensais de uma empresa — à medida que o negócio cresce, o impacto da sazonalidade também aumenta.**
- Para mais detalhes, acesse: <https://www.datacamp.com/pt/tutorial/time-series-decomposition>



EXERCÍCIOS

1. Por meio dos *datasets* **airtravel.csv** e **co2_emissions.csv**, trace suas Séries Temporais.

Em seguida, para cada uma das Séries, realize a sua decomposição e responda as seguintes perguntas:

- a. A série possui Tendência? Se sim, que tipo?
- b. A série possui Sazonalidade? Se sim, qual o período que ela acontece?
- c. A série apresenta um Ciclo? Se sim, por qual razão?

Obs: após a plotagem e decomposição das séries, responda as perguntas por meio de comentários no próprio código.



7.5. PREVISÃO DE SÉRIES TEMPORAIS

- São estimativas de valores futuros de uma série com base em seus dados históricos;
- Recurso de grande valia para auxiliar gestores na tomada de decisão;
- Exemplos de Aplicação:
 - Prever vendas (Varejo);
 - Prever ocupação de leitos (Saúde);
 - Estimar preços de ações (Finanças);
 - Prever temperatura (Clima);
 - Prever consumo de produção (Indústria);
- Podemos utilizar de modelos Estatísticos ou Baseados em *Machine Learning* para se fazer previsões. Aqui focaremos em modelos **Estatísticos**.



7.5. PREVISÃO DE SÉRIES TEMPORAIS

- A previsão não é uma verdade absoluta, pois ela pode conter erros;
- Porém, são vários os métodos que podem ser utilizados para se trabalhar com previsões de séries temporais;
- Aqui discutiremos dois deles: o **Holt-Winters** e **ARIMA**
- **Holt-Winters (Suavização Exponencial Sazonal):**
 - Capaz de detectar efeitos sazonais e tendências de uma série;
 - Também trabalha com duas variações: aditiva e multiplicativa;
 - Melhor para previsões rápidas em séries com padrões mais regulares e sazonalidade clara;
- **ARIMA**
 - Mais robusto para séries complexas e ajustável a diferentes padrões;
 - Possui diferentes variações para lidar com séries que possuem sazonalidade regular (SARIMA), variáveis externas (ARIMAX) ou quando se combina ambas (SARIMAX).

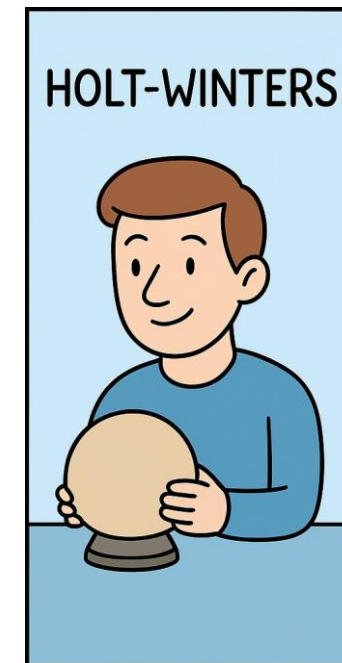


7.5. PREVISÃO DE SÉRIES TEMPORAIS

- **Holt-Winters (Suavização Exponencial Sazonal):**

- Neste método, cada novo valor previsto é uma média ponderada entre valores passados e estimativas anteriores, com pesos decrescendo exponencialmente;
- A previsão é feita utilizando 3 coeficientes responsáveis por controlarem a influência dos dados recentes na atualização dos componentes: α (nível), β (tendência) e γ (sazonalidade);
- A previsão é gerada a partir de fórmulas recursivas que atualizam os coeficientes a cada período, com base no erro entre o valor real e a previsão anterior;
- Para saber mais sobre o Holt-Winters, acesse:

<https://www.kaggle.com/code/mehmetisik/smoothing-methods-holt-winters>



7.5. PREVISÃO DE SÉRIES TEMPORAIS

- Realizando a previsão com Holt-Winters

- EXEMPLOS:

```
import pandas as pd
```

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
```

```
# importando o dataset
```

```
df = pd.read_csv('retail_index_br.csv', delimiter=';', index_col='date', parse_dates=True)
```

```
# criando um modelo de previsão
```

```
model = ExponentialSmoothing(endog=df.retail_index, trend = 'add',  
                             seasonal = 'add', seasonal_periods=12).fit()
```

```
# realizando a previsão
```

```
# limitando o gráfico para facilitar a visualização
```

```
# prevendo 3 anos
```

```
predictions = model.forecast(steps = 36)
```

```
df['retail_index']['2015-01-01:'].plot(figsize= (8, 6))
```

```
predictions.plot()
```



7.5. PREVISÃO DE SÉRIES TEMPORAIS

- **AUTOREGRESSIVE INTEGRATED MOVING AVERAGE (ARIMA):**

- O ARIMA exige que a série seja estacionária, ou seja, tenha média e variância constantes ao longo do tempo. Isso garante previsibilidade estatística;
- **Auto-Regressão (AR):** Representa a dependência linear entre o valor atual da série e seus próprios valores passados, controlada pelo parâmetro p ;
- **Integração (I):** Aplica diferenciações sucessivas (parâmetro d) à série original para remover tendências e torná-la estacionária;
- **Média Móvel (MA):** Modela a relação entre o valor atual e os erros (resíduos) de previsões anteriores, definida pelo parâmetro q ;

- Para saber mais sobre o ARIMA, acesse:

<https://www.kaggle.com/code/prashant111/arima-model-for-time-series-forecasting>



7.5. PREVISÃO DE SÉRIES TEMPORAIS

- Vai executar o ARIMA no Google Colab? Então atenção com as orientações abaixo:
 - A biblioteca **pmdarima** possui incompatibilidade com a versão 2.0 do NumPy;
 - Por isso, antes de instalar esta biblioteca, faça um *downgrade* da versão do NumPy oferecida por padrão no Google Colab:
 - **EXEMPLO:**
Setando uma versão do NumPy que o pmdarima funcione
!pip install numpy==1.23.5
 - Uma vez instalada a versão compatível, instale as bibliotecas **statsmodels** e **pmdarima**:
 - **EXEMPLO:**
!pip install statsmodels
!pip install pmdarima



7.5. PREVISÃO DE SÉRIES TEMPORAIS

- Realizando a previsão com ARIMA

- EXEMPLOS:

```
import pandas as pd
```

```
from pmdarima import auto_arima
```

```
# importando o dataset
```

```
df = pd.read_csv('retail_index_br.csv', delimiter=';', index_col='date', parse_dates=True)
```

```
# criando um modelo de previsão
```

```
# o parâmetro m indica a sazonalidade, que neste dataset acontece anualmente
```

```
model = auto_arima(y = df['retail_index'], m = 12)
```

```
# mostrando os melhores parâmetros calculados pelo auto_arima
```

```
print(model)
```

```
# realizando a previsão
```

```
predictions = pd.Series(model.predict(n_periods = 36))
```

```
df['retail_index']['2015-01-01:'].plot(figsize= (8, 6))
```

```
predictions.plot()
```



7.5. PREVISÃO DE SÉRIES TEMPORAIS

- **MAS COMO IDENTIFICAR QUAL MODELO SERIA O MELHOR PARA SE FAZER UMA PREVISÃO?**
- As vezes, uma análise exploratória da série pode ser suficiente, permitindo identificar a presença de tendência, sazonalidade, ciclos e ruído;
- Porém, uma forma mais efetiva e numérica de se fazer isso é:
 - Divida os dados históricos da série em um conjunto de treino e teste;
 - Treine os modelos para fazer previsões sobre os dados de teste, por meio dos dados de treinamento;
 - Identifique o modelo que menor erro ao se realizar as previsões sobre os dados de teste. Isso pode ser feito utilizando de métricas como:
 - MAE (Mean Absolute Error),
 - RMSE (Root Mean Squared Error) e
 - MAPE (Mean Absolute Percentage Error);
 - O modelo que apresentar menor erro, tende a ser o melhor para se realizar previsões de dados futuros.



7.5. PREVISÃO DE SÉRIES TEMPORAIS

- **HOLT-WINTERS**

- EXEMPLOS:

```
import pandas as pd
from statsmodels.tsa.holtwinters import ExponentialSmoothing
```

```
# aplicando o modelo com dados de treinamento
model = ExponentialSmoothing(endog=conjunto_treinamento.retail_index, trend =
'mul',
                             seasonal = 'mul', seasonal_periods=12).fit()
```

```
# realizando a previsão no mesmo tamanho dos dados de teste
predictions = model.forecast(steps = len(conjunto_teste))
```

```
# mostrando dados de treinamento (de 2015 em diante para facilitar a visualização)
df['retail_index']['2015-01-01:'].plot(figsize= (8, 6))
```

```
# mostrando dados de teste
conjunto_teste['retail_index'][:].plot()
```

```
# mostrando previsões
predictions.plot(legend = True, label='Previsão')
```



7.5. PREVISÃO DE SÉRIES TEMPORAIS

- **ARIMA**

- Identificando o modelo ARIMA ideal para os dados de treinamento:

```
from pmdarima.arima import auto_arima
```

```
# aplicando o modelo com dados de treinamento
```

```
model = auto_arima(y = conjunto_treinamento['retail_index'], m = 12)
```

```
# mostrando o resultado do melhor modelo ARIMA detectado
```

```
print(model)
```

- Realizando a previsão sobre os dados de teste:

```
# realizando as previsoes
```

```
predictions = pd.Series(model.predict(n_periods=len(conjunto_teste)))
```

```
# mostrando dados de treinamento
```

```
conjunto_treinamento['retail_index']['2015-01-01:'].plot(figsize= (8, 6))
```

```
# mostrando dados de teste
```

```
conjunto_teste['retail_index'][:].plot()
```

```
# mostrando previsões
```

```
predictions.plot(legend = True, label='Previsão')
```



7.5. PREVISÃO DE SÉRIES TEMPORAIS

- O Erro Percentual Absoluto Médio (MAPE, do inglês *Mean Absolute Percentage Error*) é um dos critérios mais populares utilizados para auxiliar na seleção de modelos;
- Quanto menor o erro produzido pelo modelo, melhor;
- EXEMPLOS:

```
import numpy as np
```

```
# Função do MAPE
```

```
def MAPE(y_true, y_pred):
```

```
    y_true, y_pred = np.array(y_true), np.array(y_pred)
```

```
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100
```

```
# Cálculo dos MAPEs
```

```
print('MAPE Holt-Winters:', MAPE(conjunto_teste['retail_index'],  
forecasting_hw))
```

```
print('MAPE ARIMA:', MAPE(conjunto_teste['retail_index'],  
forecasting_arima))
```



inatel.tecnologias 
inatel.tecnologias 
inateloficial 
company/inatel 
www.inatel.br 

Campus em Santa Rita do Sapucaí
Minas Gerais - Brasil
Av. João de Camargo, 510
Centro - 37536-001

CIÊNCIA DE DADOS COM PYTHON

FIM CAPÍTULO 7

Inatel
_o futuro
não tem hora,
mas tem lugar.